ACTL3142 Tutorial

Week 9: Moving Beyond Linearity

Nick Guo

School of Risk and Actuarial Studies, UNSW Sydney

T1 2026

Recap

From Week 8 (Tree-Based Methods):

- **Decision trees**: partition the predictor space into rectangular regions, predict a constant (mean or majority class) in each.
- **Bagging/Random forests**: average many deep trees to reduce variance.
- **Boosting**: combine many stumps sequentially to reduce bias.

New direction: trees abandon the regression framework entirely. This week, we stay within regression but **relax the linearity assumption**. The key idea: transform the predictors before fitting.

Today: **Splines** and **Generalised Additive Models (GAMs)**.

The Basis Function Idea

Every method this week fits the same type of model:

$$y_i = \beta_0 + \sum_{k=1}^{K} \beta_k \, b_k(x_i) + \varepsilon_i$$

where $b_1(x), b_2(x), \ldots, b_K(x)$ are **basis functions**: fixed, known transformations of x.

The model is still **linear in** β, so all our OLS theory applies. The "nonlinearity" comes entirely from the choice of basis functions.

Linear in what?

The model is non-linear in x but linear in the parameters $\beta_0, \beta_1, \ldots, \beta_K$. We can still use `lm()`, compute standard errors, do hypothesis tests, etc.

Different basis choices give polynomial regression, step functions, and splines. The question is: which transformation captures the pattern best?

Polynomial Regression

The simplest extension: add powers of x as extra predictors.

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_d x_i^d + \varepsilon_i$$

Basis functions: $b_j(x) = x^j$ for $j = 1, \ldots, d$. Degree d is a hyperparameter (typically 2–4, chosen via CV).

In R: `lm(y ~ poly(x, d))` uses **orthogonal polynomials** for numerical stability (the raw monomials $x, x^2, \ldots$ become highly correlated at high d).

Pros: simple, just extends `lm()`. Inference is straightforward.

Cons: global structure. Every data point influences the fit everywhere, and the curve can behave erratically at the boundaries for large d.

`poly()` uses a rotated basis spanning the same space as raw monomials. Predictions $\hat{y}$ are identical, but $X^\top X$ is well-conditioned for stable computation.

Step Functions

The opposite extreme: assume the relationship is **constant within bins**.

Break the range of x into $K+1$ regions using cutpoints $c_1 < c_2 < \cdots < c_K$:

$$y_i = \beta_0 + \beta_1 I(c_1 \leq x_i < c_2) + \cdots + \beta_K I(c_K \leq x_i) + \varepsilon_i$$

Basis functions: $b_j(x) = I(c_j \leq x < c_{j+1})$ (indicator functions).

In R: `lm(y ~ cut(x, breaks = K))`

Pros: fully **local** (each bin is independent of the others).

Cons: discontinuous at the cutpoints. Ignores ordering within each bin.

Polynomials vs step functions: two extremes

Polynomials are smooth but global (one function everywhere). Step functions are local but discontinuous. Splines combine the best of both.

From Piecewise Polynomials to Splines

Idea: fit separate polynomials in different regions, but require them to connect smoothly at the boundaries (**knots**).

A **spline** of degree d with knots $\xi_1, \ldots, \xi_K$:

- A piecewise polynomial of degree d in each interval between knots
- Continuous up to the $(d-1)$th derivative at each knot

Cubic spline ($d = 3$) is the most common choice:

- Continuous function, continuous 1st and 2nd derivatives at each knot
- The lowest degree where the knots are **invisible to the eye**
- Degrees of freedom: $K + 3$ (with K knots)

In R: `lm(y ~ bs(x, knots = c(...), degree = 3))` from the `splines` package.

Counting: $4(K+1) - 3K = K + 4$ *total parameters (with intercept);* `bs()` *produces* $K + 3$ *columns since* `lm()` *supplies the intercept.*

Natural Splines and Choosing Knots

Problem: cubic splines can behave erratically at the **boundaries** (extrapolation beyond the outermost knots).

Natural spline: a cubic spline forced to be **linear** beyond the boundary knots. This adds 4 extra constraints (2 at each end), freeing up df for the interior.

In R: `ns(x, df = ...)` or `ns(x, knots = c(...))` from the `splines` package.

Choosing knots: place at **quantiles** of x (uniform in data density). Select the number of knots via CV.

Regression splines vs smoothing splines

Regression splines require you to choose the number *and* location of knots. Smoothing splines avoid this by placing a knot at every data point and penalising roughness instead.

Smoothing Splines

Instead of choosing knots, solve an **optimisation problem**:

$$\min_{g} \quad \frac{1}{n}\sum_{i=1}^{n}(y_i - g(x_i))^2 \; + \; \lambda \int g''(x)^2 \, dx$$

First term: fit the data. Second term: penalise **roughness** ($g'' = 0$ is a straight line; more curvature = larger penalty). λ controls the tradeoff.

Extremes: $\lambda = 0 \Rightarrow g$ interpolates every point. $\lambda \to \infty \Rightarrow g$ becomes a straight line.

Despite searching over all smooth functions, the minimiser is provably a **natural cubic spline** with a knot at every x_i. The spline form is a consequence of the penalty, not an assumption.

Same idea as ridge/lasso: penalise complexity and tune by CV. Here λ penalises curvature instead of coefficient size.

Local Regression

Idea: at each target point x_0, fit a **weighted least squares** regression using only nearby data.

Algorithm:

1. Gather the fraction $s = k/n$ of training points closest to x_0
2. Assign weights to those k points: $K_{i0} > 0$, decreasing with distance from x_0; all other points have $K_{i0} = 0$
3. Fit weighted least squares over those k points: $\min_\beta \sum_{i \in \mathcal{N}(x_0)} K_{i0}(y_i - \beta_0 - \beta_1 x_i)^2$
4. Predict: $\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0$

Span s controls flexibility: small s = flexible, large s = smooth. In R: `loess(y ~ x, span = s)`.

Works best with 1–2 predictors. In higher dimensions, the "curse of dimensionality" means too few local neighbours. For multiple predictors, use GAMs.

Generalised Additive Models (GAMs)

All the methods so far handle **one predictor**. For multiple predictors, replace each linear term with a flexible function:

$$y_i = \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \cdots + f_p(x_{ip}) + \varepsilon_i$$

Each f_j can be any method: smoothing spline, natural spline, polynomial, or just linear. Fitted via **backfitting**: at each step, compute partial residuals $r_i = y_i - \hat{\beta}_0 - \sum_{k \neq j} \hat{f}_k(x_{ik})$ and fit $\hat{f}_j$ to those residuals. Cycle through all j until convergence.

In R: `gam(y ~ s(x1) + s(x2) + x3)` from `mgcv`. `s()` fits a smoothing spline; `x3` enters linearly.

GAMs vs trees

Trees capture **interactions** automatically but are harder to interpret per-variable. GAMs assume **additivity** (no interactions unless explicitly added) but give clear partial effect plots.

GAMs extend to classification:
$\log\left(\frac{p(\mathbf{x})}{1-p(\mathbf{x})}\right) = \beta_0 + f_1(x_1) + \cdots + f_p(x_p).$